2nd June 2026

second assignment

Team

Martim Santos, 114614
Sebastião Teixeira, 114624
Duarte Santos, 113304
Henrique Teixeira, 114588

Subject

AS

2nd June 2026

Checkpoint

The problem

Scenario C

Omnichannel Commerce Core

Buy-Online, Fulfill Elsewhere	Cross-Channel Stock	Stay Useful Under Failure
An order placed on the web store must reach the ERP and the warehouse, reliably, even when those systems fail.	A sale in the physical store must update web-store stock within seconds, preventing overselling across channels.	When the WMS goes down, orders keep flowing. Degradation is visible and the system reconciles on recovery.

The hard part is not connecting systems.
It is staying useful, visible, and recoverable when a dependency fails.

What We Built

Commerce Core nopCommerce

- Transactional Outbox
- OutboxPublisher background service
- Stock Update consumer with OSPOS priority conflict resolution
- /integration/health endpoint

Integration & Messaging

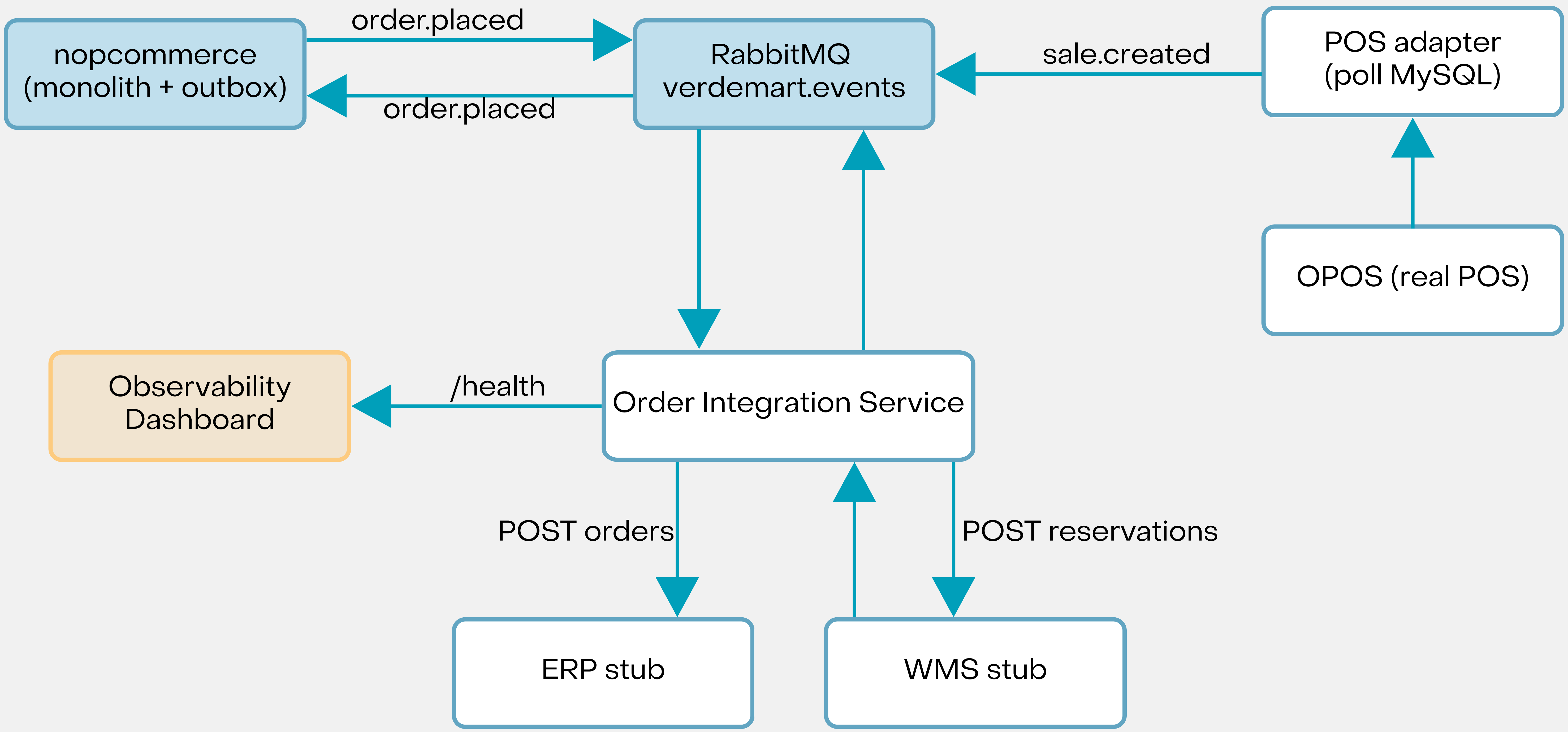
- RabbitMQ verdemart.events + verdemart.dlx
- Order Integration Service
- Polly Retry (ERP) + Circuit Breaker (WMS)
- Reconciliation loop drains the DLQ

External Systems & Observability

- ERP +WMS stubs with live failure injection
- Real OSPOS POS + .NET polling adapter
- Liver Observability Dashboard

One Docker Compose

What We Built – Architecture

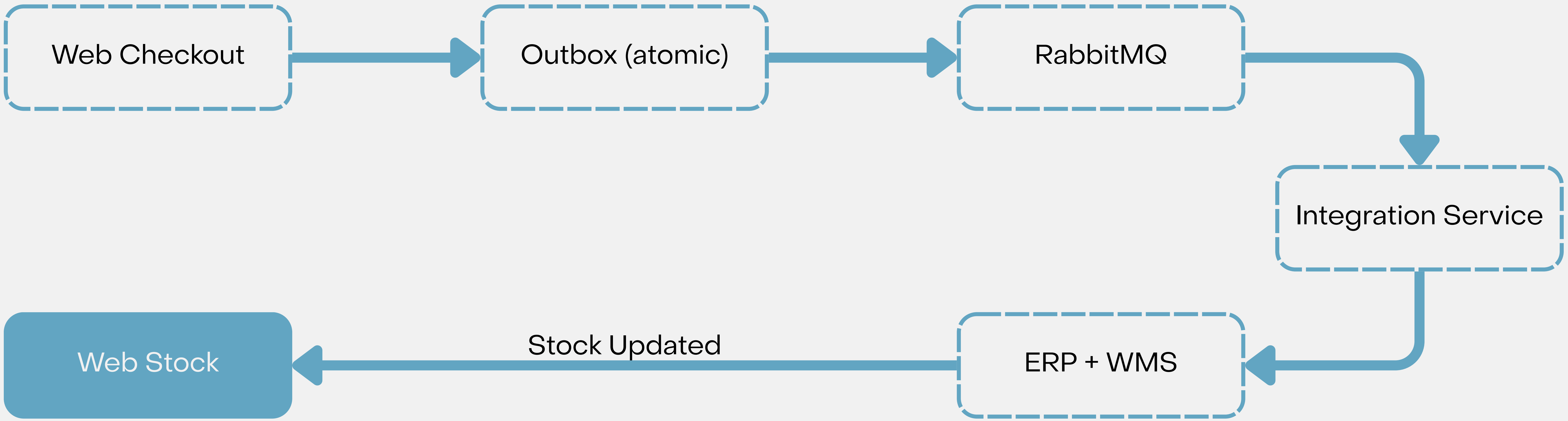


2nd June 2026

Use Cases

UC1 – Buy Online, fulfill through another channel

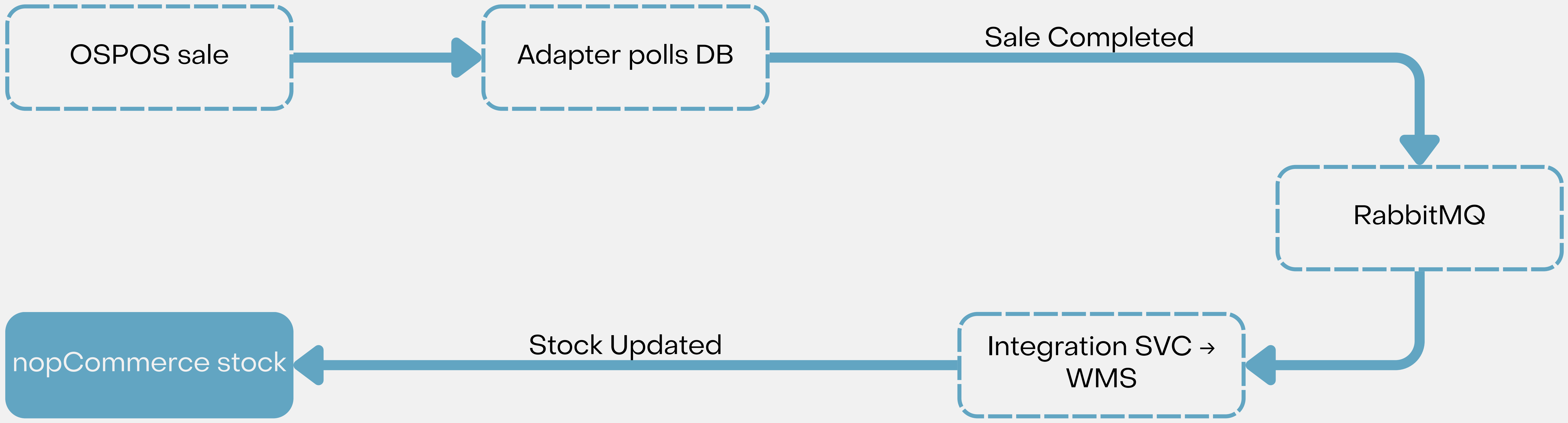
An online order is confirmed instantly, then flows asynchronously through the outbox and broker to the Integration Service, which fulfills it via ERP and WMS, and the resulting stock correction flows back to the web store.



UC1 – Buy Online, fulfill through another channel – Live Demo

UC2 - Cross-channel stock visibility

A physical-store sale in OSPOS is picked up by the adapter polling its database, published as sale.completed, and reconciled through the WMS so the new stock level becomes visible on the web store within seconds.



UC2 - Cross-channel stock visibility - Live Demo

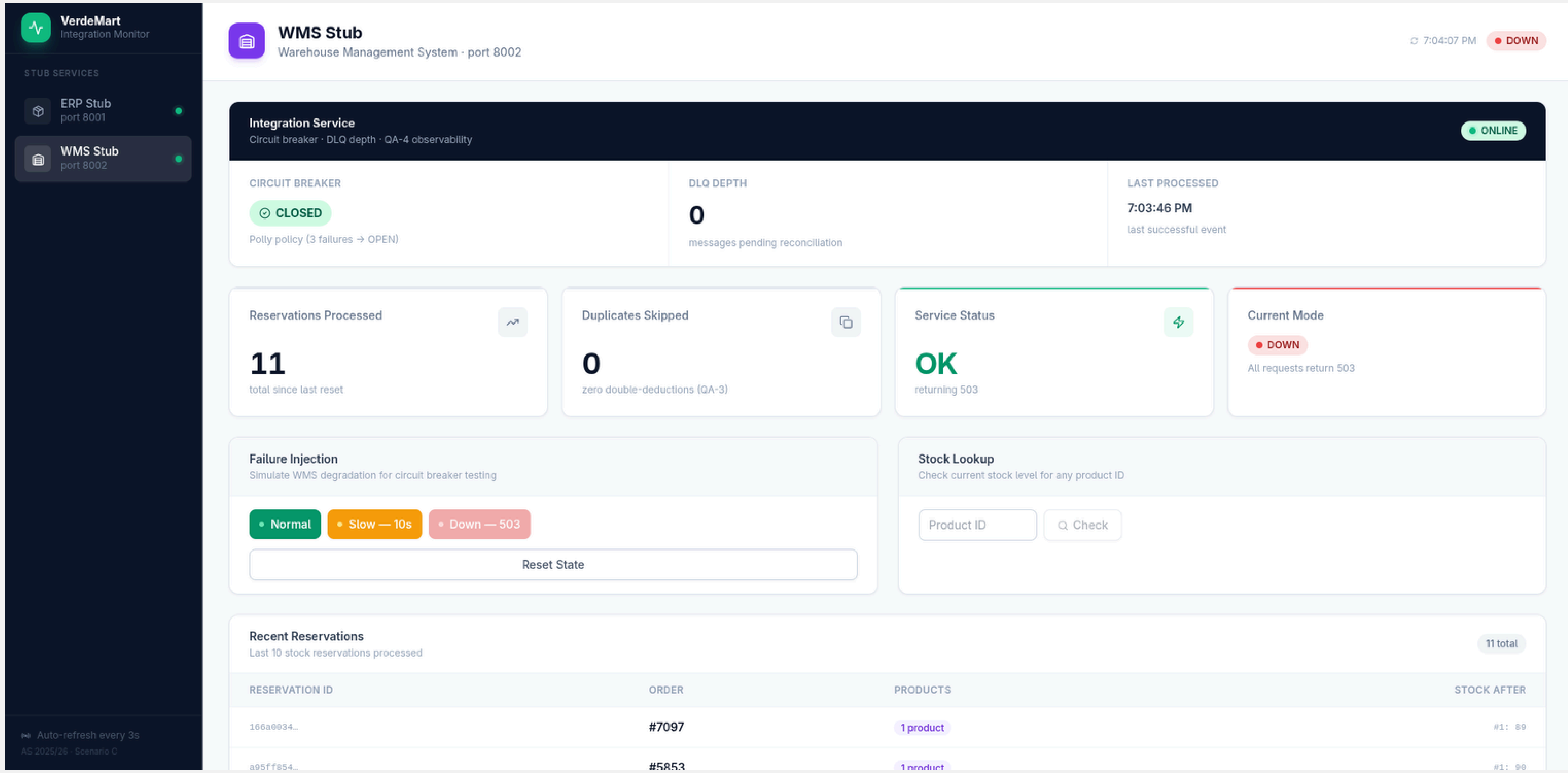
Behavior Under Degradation

Operator View

Circuit OPEN after 3 failures goes to DLQ
DLQ depth climbs in real time
WMS shown DOWN, visible < 5s

Customer Experience

Checkout succeeds, confirmed instantly
Outbox decouples it from the failing WMS
0% orders lost during the outage



UC3 – WMS goes down

Place Orders

Customers check out orders confirmed instantly via the outbox

WMS goes DOWN

Flipped from the dashboard. Reservations fail, the circuit breaker OPENS

Degradation visible

Dashboard shows circuit OPEN and the DLQ depth climbing

WMS goes NORMAL

Reconciliation drains the DLQ, stock flows, the circuit CLOSES

What you will see?

- 0% of orders blocked during the outage;
- DLQ fully drained < 60 s after recovery;
- Zero duplicate reservations;
- Web-store stock corrected automatically on recovery.

UC3 – WMS goes down – Live Demo

Reliability Mechanisms

	Where	What it handles
Transactional Outbox	nopCommerce	At-least-once delivery even if RabbitMQ is temporarily down
Retry + backoff	ERP adapter (Polly)	Transient ERP 503s — 3 attempts, exponential backoff
Circuit breaker	WMS adapter (Polly)	Prolonged WMS outage — opens after 3 consecutive failures
Dead-letter queue	RabbitMQ verdemart.dlx	Preserves undeliverable reservations during the outage
Reconciliation loop	Integration Service	Drains the DLQ on recovery and resubmits to the WMS
Idempotency on eventId	Consumers	No duplicate ERP orders or stock adjustments on replay

Evidence Pack

	Target	Result (verified in live demo)
QA-1	Availability — 0% orders blocked; queued ≤ 60 s after recovery	Met — orders confirmed throughout the WMS outage
QA-2	Consistency — web stock updated < 30 s of WMS / POS event	Met — store sale reflected on the web store in-window
QA-3	Recoverability — DLQ drained < 60 s; no duplicates	Met — DLQ drained on recovery, idempotent replay
QA-4	Observability — circuit state visible < 5 s	Met — dashboard shows OPEN on first timeout
QA-5	Reliability — ERP recovers via retry < 30 s	Met — order lands on 3rd attempt, no loss

Failure scenarios exercised: WMS permanently down · ERP 503 ×2 then recover · nopCommerce restart mid-flush · POS sale vs web order on the last unit.

Architecture Decisions — As Built

	Decision	As-built outcome
ADR-01	RabbitMQ over Kafka	Native DLX, zero ops overhead — verdemart.dlx backs the WMS DLQ
ADR-02	Outbox over direct publish	IntegrationEvent written in the order transaction — no lost events
ADR-03	.NET worker for Integration Service	Polly circuit breaker + retry are first-class — used as designed
ADR-04	Stubs for ERP/WMS, real OSPOS	Stubs inject failure on demand; OSPOS is real, bridged by an adapter

2nd June 2026

Limitations

Tradeoffs & Known Limitations

Limitations

01 Outbox polling adds ~3 s latency

02 Stubs hold state in memory

03 OSPOS polling 30–60 s lag

04 ERP has no circuit breaker / DLQ

05 Dashboard polls every 2 s

Last Unit Race: Web vs OSPOS

Question

Stock=1

At the same time:
- A web customer is in checkout
- A customer is carrying the last unit

Both flows beli

What happens?

Requirements

Never refuse a physical sale.

Proposed solution

Dashboard shows circuit OPEN and the DLQ depth climbing

any questions?